

Clase 032 — Nmap Scripting Engine (NSE)

Parte: 1 — Redes y seguridad de redes · Fuente: *Nmap Network Scanning, G. Lyon*  Duración estimada: 110 min · Nivel: Intermedio

Objetivo

Aprovechar el **Nmap Scripting Engine** para automatizar tareas de descubrimiento, detección de vulnerabilidades, enumeración y hasta explotación ligera mediante scripts en Lua. El alumno aprenderá a elegir scripts por categoría, pasarles argumentos, leer su salida y escribir un script mínimo propio.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Seleccionar** scripts por nombre, categoría o expresión.
2. **Ejecutar** el conjunto por defecto (`-sC`) y conjuntos temáticos (vuln, discovery, safe).
3. **Pasar** argumentos a los scripts con `--script-args` .
4. **Actualizar** la base de datos de scripts y consultar su documentación local.
5. **Interpretar** con criterio la salida de scripts de vulnerabilidad.
6. **Escribir** un script NSE elemental en Lua.

Temas

#	Tema	Por qué importa
1	Categorías NSE (safe, default, vuln, exploit...)	Elegir según objetivo y riesgo
2	Selección de scripts (<code>--script</code>)	Precisión en la automatización
3	Argumentos (<code>--script-args</code>)	Adaptar scripts al contexto
4	<code>-sC</code> y <code>-A</code>	Conjuntos convenientes
5	<code>--script-help</code> y <code>--script-updatedb</code>	Documentación y mantenimiento
6	Scripts de vulnerabilidad y su fiabilidad	Evitar falsos positivos
7	Estructura de un script Lua	Extender Nmap

Definiciones y características

- **NSE:** motor que ejecuta scripts Lua durante o después del escaneo, con acceso a los resultados de puertos y a librerías de red.
- **Categoría:** etiqueta que agrupa scripts por propósito y riesgo: `safe` , `default` , `discovery` , `version` , `auth` , `brute` , `vuln` , `exploit` , `intrusive` , `dos` , `malware` .
- `--script-args` : mecanismo para pasar parámetros (credenciales, rutas, límites) a los scripts.

- **Script de vulnerabilidad:** comprueba una debilidad concreta (p. ej. `ssl-heartbleed`) y reporta estado, referencias y a veces CVE.
- `--script-help`: muestra la documentación de un script sin ejecutarlo.

Herramientas y preparación

- **Nmap 7.x** (incluye ~600 scripts NSE en `/usr/share/nmap/scripts/`).
- Actualiza la base de datos:

```
bash sudo nmap --script-updatedb
```

- Objetivos de laboratorio: un servidor web, un SMB, un servicio con TLS. Contenedores deliberadamente vulnerables (aislados) para practicar scripts `vuln`.

⚠ **Nota ética:** las categorías `intrusive`, `brute`, `exploit` y `dos` pueden dañar o bloquear servicios y realizan acciones ofensivas. Úsalas **solo** en sistemas propios o con autorización explícita y alcance definido. Nunca contra terceros.

Laboratorio guiado

1. **Scripts por defecto** (categoría `default`, seguros y útiles):

```
bash sudo nmap -sC 192.168.56.101
```

2. **Enumeración HTTP:**

```
bash sudo nmap -p80 --script http-title,http-headers,http-enum 192.168.56.101
```

3. **Enumeración SMB:**

```
bash sudo nmap -p445 --script smb-os-discovery,smb-enum-shares 192.168.56.101
```

4. **TLS/certificados:**

```
bash sudo nmap -p443 --script ssl-cert,ssl-enum-ciphers 192.168.56.101
```

5. **Categoría vuln** (solo en tu laboratorio):

```
bash sudo nmap -sV --script vuln 192.168.56.101
```

6. **Pasar argumentos** (ejemplo con enumeración HTTP y user-agent):

```
bash sudo nmap -p80 --script http-enum --script-args http.useragent="Lab-Scanner" 192.168.56.101
```

7. **Consultar ayuda** de un script:

```
bash nmap --script-help ssl-enum-ciphers
```

8. **Selección por expresión** (todos los `http-*` menos los intrusivos):

```
bash sudo nmap -p80 --script "http-* and not intrusive" 192.168.56.101
```

9. Escribe un script NSE mínimo `hello.nse` :

```
lua description = "Devuelve un saludo por cada puerto abierto" author = "alumno" categories = {"safe"} portrule = function(host, port) return port.state == "open" end action = function(host, port) return "Hola desde el puerto " .. port.number end
```

Ejecútalo:

```
bash sudo nmap -p80 --script ./hello.nse 192.168.56.101
```

Ejercicios

1. Lista todos los scripts de la categoría `vuln` con `ls /usr/share/nmap/scripts/ | grep vuln` y elige tres para leer su `--script-help`.
2. Ejecuta `-sC` y explica qué información añadió cada script a la salida.
3. Usa `ssl-enum-ciphers` y evalúa si el objetivo soporta cifradores débiles.
4. Modifica `hello.nse` para que también imprima el nombre del servicio detectado.
5. Combina `-sV` con `--script banner` y compara con la detección de versión nativa.
6. Investiga la diferencia entre un `portrule` y un `hostrule` en NSE.

Reto verificable

Escribe (o adapta) un escaneo NSE que enumere un servicio web de tu laboratorio y produzca: título de la página, cabeceras de seguridad presentes/ausentes y cifradores TLS soportados. Entrega el comando, la salida `-oN` y una breve interpretación de dos hallazgos de seguridad.

Criterio de aceptación: la salida incluye los tres bloques de información y la interpretación identifica correctamente al menos una debilidad real (p. ej. falta de HSTS o soporte de un cifrador obsoleto).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"'X' did not match a category, filename, or directory"	Nombre de script mal escrito o base sin actualizar; corre <code>--script-updatedb</code>
Script no devuelve nada	El <code>portrule</code> no se cumple (puerto cerrado o servicio distinto); verifica con <code>-sV</code>
Escaneo <code>vuln</code> tumba el servicio	Usaste scripts intrusivos; limita a <code>safe / default</code> fuera del laboratorio
Argumentos ignorados	Formato incorrecto de <code>--script-args</code> ; usa <code>clave=valor</code> separados por coma
Salida de <code>brute</code> sin resultados	Diccionario inadecuado o cuenta bloqueada; ajusta <code>userdb / passdb</code> (solo en laboratorio)

? Preguntas frecuentes

? ¿`-sC` es lo mismo que `--script default`? Sí. `-sC` ejecuta todos los scripts de la categoría `default`, considerados seguros y útiles para un escaneo general.

? ¿Los scripts `vuln` explotan la vulnerabilidad? Normalmente solo la detectan. Los que explotan están en la categoría `exploit` y son mucho más peligrosos; úsalos con extremo cuidado y solo con permiso.

? ¿En qué lenguaje se escriben los scripts? En Lua, usando las librerías NSE (nmap, shortport, http, etc.). Son fáciles de leer y modificar.

? ¿Cómo sé qué hace un script antes de ejecutarlo? Con `nmap --script-help <nombre>`, que muestra descripción, categorías, argumentos y ejemplos sin lanzar nada.

Referencias

- Lyon, G. *Nmap Network Scanning*, cap. "Nmap Scripting Engine". <https://nmap.org/book/nse.html>
- NSE Script Documentation. <https://nmap.org/nsedoc/>
- NSE: Writing Scripts. <https://nmap.org/book/nse-tutorial.html>
- Categorías NSE. <https://nmap.org/book/nse-usage.html>

Siguiendo clase

Clase 033 - Enumeracion de servicios de red